

Data Structure and Algorithm

(PMSCS 623P)

Lecture 2

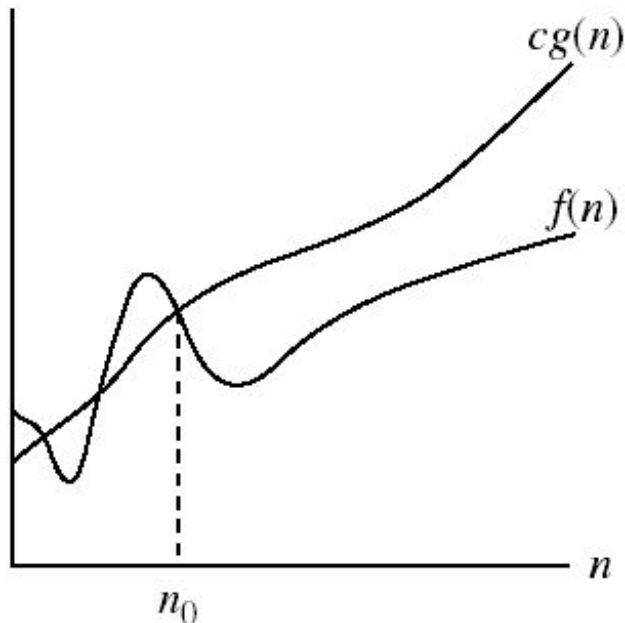
Asymptotic Notations

- O notation: asymptotic “upper bound”:
- Ω notation: asymptotic “lower bound”:
- Θ notation: asymptotic “tight bound”:

Asymptotic Notations

- *O-notation (Big Oh)*

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$



$O(g(n))$ is the set of functions with smaller or same order of growth as $g(n)$

Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $O(n^2)$, $O(n^2 \lg n)$, $O(n^3)$, $O(n^4)$, ...

$O(n^2 \lg n)$, $O(n^3)$, $O(n^4)$,

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is.

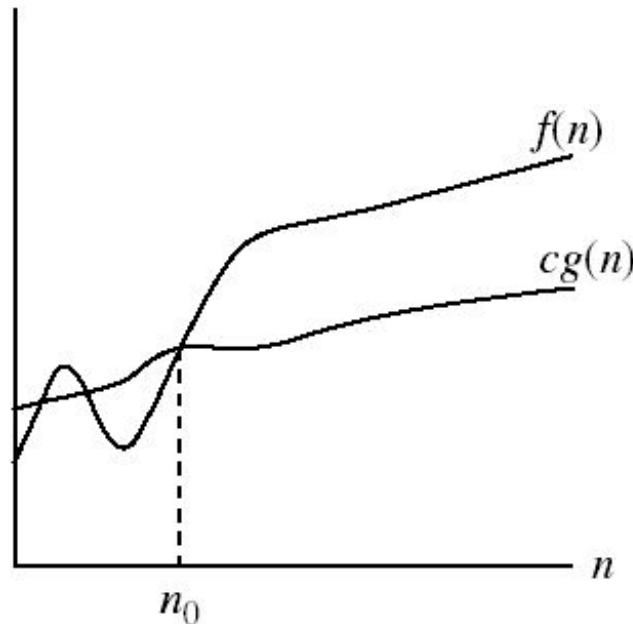
$g(n)$ is an *asymptotic upper bound* for $f(n)$.

Loose upper bounds

Asymptotic Notations

- Ω - notation (*Big Omega*)

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$.



$\Omega(g(n))$ is the set of functions with larger or same order of growth as $g(n)$

Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $\Omega(n^2)$, $\Omega(n \lg n)$, $\Omega(n)$, $\Omega(\lg n)$,

$\Omega(1)$

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $\Omega(n^2 \lg n)$, $\Omega(n^2)$, $\Omega(n)$, ...

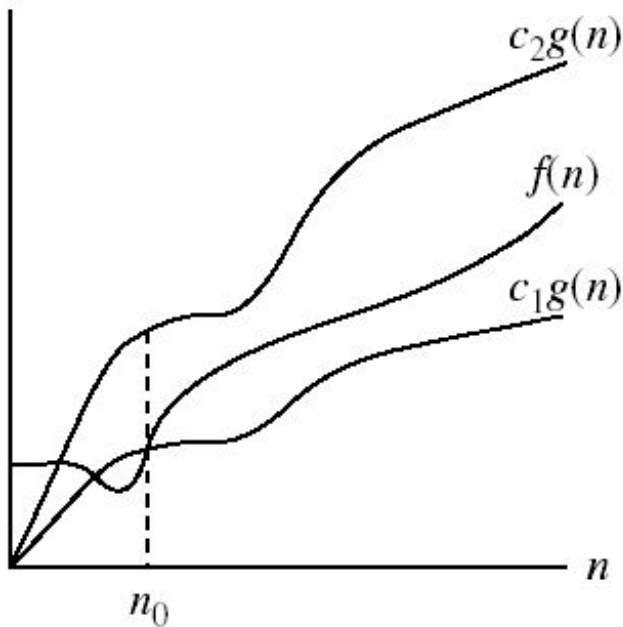
$g(n)$ is an *asymptotic lower bound* for $f(n)$.

Asymptotic Notations

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\} .$

$\Theta(g(n))$ is the set of functions with the same order of growth as $g(n)$

* $f(n)$ is both $O(g(n))$ & $\Omega(g(n)) \leftrightarrow f(n)$ is $\Theta(g(n))$



Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $\Theta(n^2)$

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $\Theta(n^2 \lg n)$

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

Big O

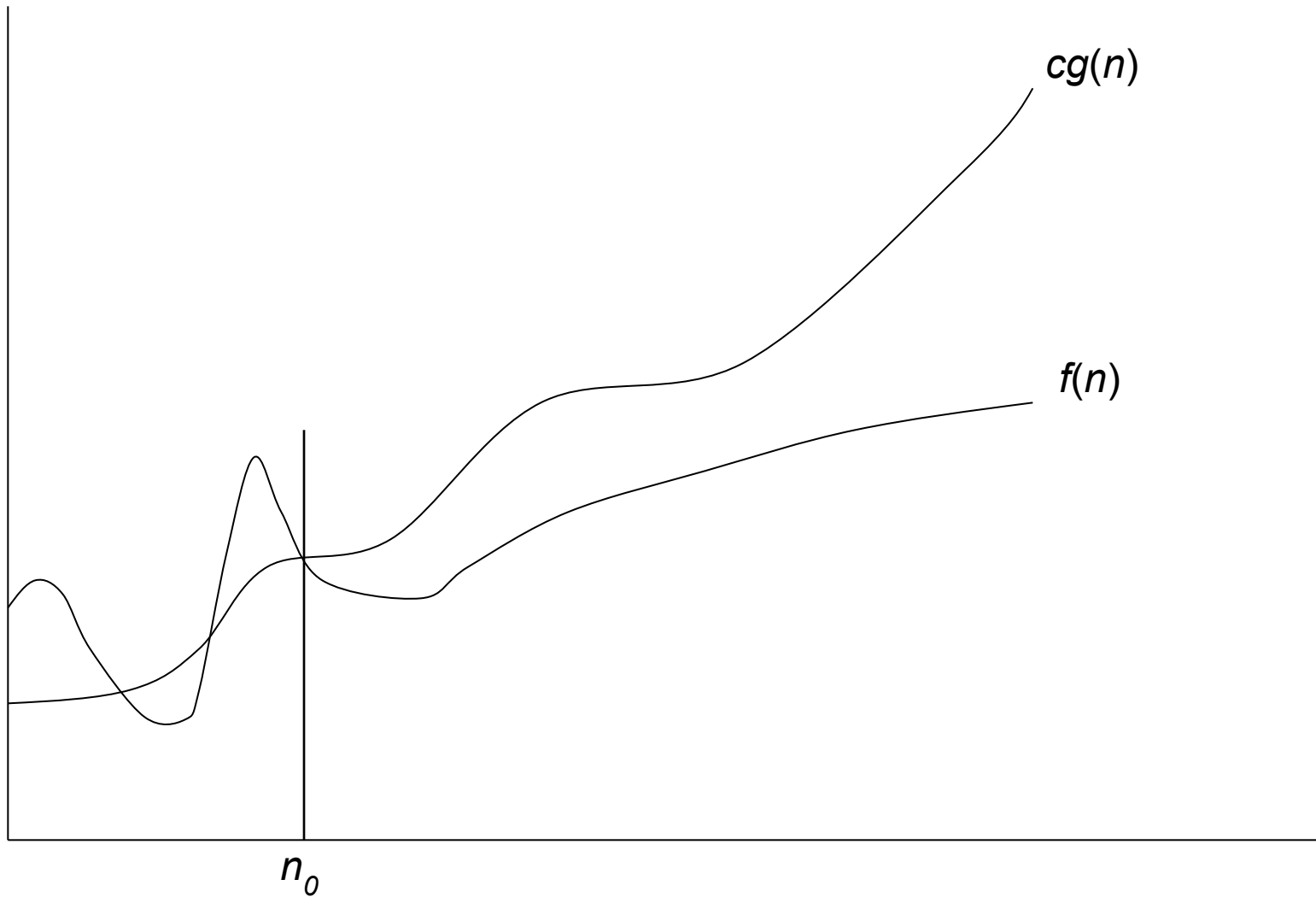
- Informally, $O(g(n))$ is the set of all functions with a smaller or same order of growth as $g(n)$, within a constant multiple
- If we say $f(n)$ is in $O(g(n))$, it means that $g(n)$ is an **asymptotic upper bound** of $f(n)$
 - Intuitively, it is like $f(n) \leq g(n)$
- What is $O(n^2)$?
 - The set of all functions that grow slower than or in the same order as n^2

Big-O

$f(n) = O(g(n))$: { there exist positive constants c and n_0 such that
 $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$ }

- What does it mean?
 - If $f(n) = O(n^2)$, then $f(n)$ can be larger than n^2 sometimes, **but...**
 - We can choose some constant c and some value n_0 such that for **every** value of n larger than n_0 : $f(n) \leq cn^2$
 - That is, for values larger than n_0 , $f(n)$ is never more than a constant multiplier greater than n^2
 - Or, in other words, $f(n)$ does not grow more than a constant factor faster than n^2

Visualization Big-O

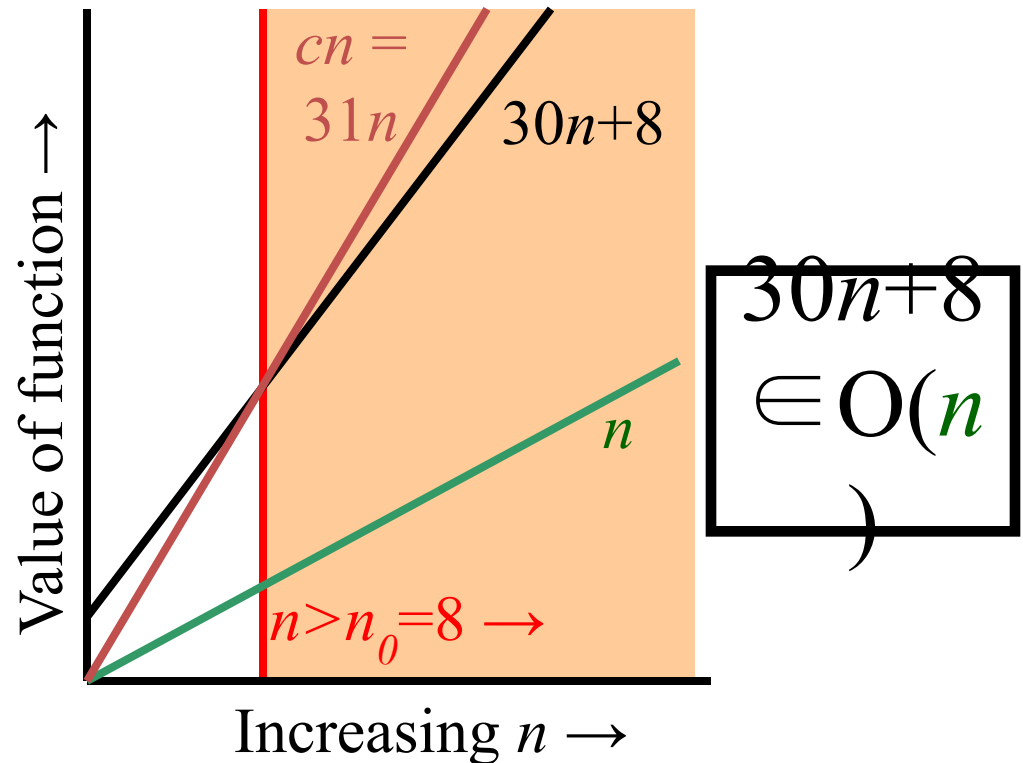


Examples

- Show that $30n+8$ is $O(n)$.
 - Show $\exists c, n_0: 30n+8 \leq cn, \forall n \geq n_0$.
 - Let $c=31, n_0=8$
 $cn = 31n = 30n + n \geq 30n+8,$
 - so $30n+8 \leq cn$.

Big-O example, graphically

- Note $30n+8$ isn't less than n *anywhere* ($n>0$).
- It isn't even less than $31n$ *everywhere*.
- But it *is* less than $31n$ everywhere to the right of $n=8$.



Example: Exponential-Time Algorithm

- Suppose we have a problem that, for an input consisting of n items, can be solved by going through 2^n cases
- We use a **supercomputer**, that analyses 200 million cases per second
 - Input with 15 items — 163 microseconds
 - Input with 30 items — 5.36 seconds
 - Input with 50 items — more than two months
 - Input with 80 items — 191 million years

Example: Quadratic-Time Algorithm

- Suppose solving the same problem with another algorithm will use $300n^2$ clock cycles on a Handheld PC, running at 33 MHz
 - Input with 15 items — 2 milliseconds
 - Input with 30 items — 8 milliseconds
 - Input with 50 items — 22 milliseconds
 - Input with 80 items — 58 milliseconds
- Therefore, to speed up program, don't simply rely on the raw power of a computer
 - Very important to use an efficient algorithm

- ❑ Once upon a time there was an Indian king who wanted to reward a wise man for his excellence. The wise man asked for nothing but some wheat that would fill up a chess board.



- ❑ But here were his rules: in the first tile he wants 1 grain of wheat, then 2 on the second tile, then 4 on the next one...each tile on the chess board needed to be filled by double the amount of grains as the previous one. The naïve king agreed without hesitation, thinking it would be a trivial demand to fulfill, until he actually went on and tried it...
- ❑ So how many grains of wheat does the king owe the wise man? We know that a chess board has 8 squares by 8 squares, which totals 64 tiles, so the final tile should have 2^{64} grains of wheat. If you do a calculation online, you will end up getting 1.8446744×10^{19} , that is about 18 followed by 18 zeroes. Assuming that each grain of wheat weighs 0.01 grams, that gives us 184,467,440,737 tons of wheat. And 184 billion tons is quite a lot, isn't it?

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

Linear search

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

Linear search


```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

Linear search

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

0	1	2	3	4	5	6	7	8
S	a	n		D	i	e	g	o

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```


How many operations get executed?

For a single iteration:

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

How many operations get executed?

For a single iteration:

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

3

How many iterations of loop?

Search for the letter
in the word

'x'

"San Diego"

```
public static boolean hasLetter (String word, char letter)
{
    for (int i = 0; i < word.length(); i++)
    {
        if (word.charAt(i) == letter)
        {
            return true;
        }
    }
    return false;
}
```

9 iterations
So
30 Operations

```
if (word.charAt(i) == letter)
{
    return true;
}
```

```
int count = 0;
for (int i = 0; i < word.length(); i++)
{
    count ++;
}
```

```
int count = 0;
for (int i = 0; i < word.length() i++)
{
    count ++;
}
```



INPUT of SIZE n

```
int count = 0;  
for (int i = 0; i < word.length(); i++)  
{  
    count ++;  
}
```

$$3n + 3$$

```
public static void reduce (int[] vals) {  
    int minIndex = 0;  
    for (int i=0; i < vals.length; i++) {  
        if (vals[i] < vals[minIndex] ) {  
            minIndex = i;  
        }  
    }  
    int minVal = vals[minIndex];  
    for (int i=0; i < vals.length; i++) {  
        vals[i] = vals[i] - minVal;  
    }  
}
```



```
public static void reduce (int[] vals) {  
    int minIndex = 0;  
    for (int i=0; i < vals.length; i++) {  
        if (vals[i] < vals[minIndex] ) {  
            minIndex = i;  
        }  
    }  
    int minVal = vals[minIndex];  
    for (int i=0; i < vals.length; i++) {  
        vals[i] = vals[i] - minVal;  
    }  
}
```

```
public static void reduce (int[] vals) {  
    int minIndex = 0; O(1)  
    for (int i=0; i < vals.length; i++) {  
        if (vals[i] < vals[minIndex] ) {  
            minIndex = i;  
        }  
    }  
    int minVal = vals[minIndex]; O(1)  
    for (int i=0; i < vals.length; i++) {  
        vals[i] = vals[i] - minVal;  
    }  
}
```

- These don't
depend on the
input size (n)

```
for (int i=0; i < vals.length; i++) {  
    if (vals[i] < vals[minIndex] ) {  
        minIndex = i;  
    }  
}
```

```
for (int i=0; i < vals.length; i++) {  
    if (vals[i] < vals[minIndex] ) {  
        minIndex = i;  
    }  
}
```

- There will be n loop iterations
- Each iteration will take constant time

```
for (int i=0; i < vals.length; i++) {  
    if (vals[i] < vals[minIndex] ) {  
        minIndex = i;  
    }  
}
```

O(n)

```
public static void reduce (int[] vals) {
```

$O(1)$

```
    for (int i=0; i < vals.length; i++) {  
        if (vals[i] < vals[minIndex] ) {  
            minIndex = i;  
        }  
    }
```

$O(n)$

$O(1)$

```
    for (int i=0; i < vals.length; i++) {  
        vals[i] = vals[i] - minVal;  
    }
```

```
}
```

```
public static void reduce (int[] vals) {
```

$O(1)$

$O(n)$

$O(1)$

```
    for (int i=0; i < vals.length; i++) {  
        vals[i] = vals[i] - minVal;
```

```
    }
```

```
}
```

```
for (int i=0; i < vals.length; i++) {  
    vals[i] = vals[i] - minVal;  
}
```


- Again, there will be n loop iterations
- Each iteration will take constant time

```
for (int i=0; i < vals.length; i++) {  
    vals[i] = vals[i] - minVal;  
}
```

O(n)

```
public static void reduce (int[] vals) {
```

$O(1)$

$O(n)$

$O(1)$

```
    for (int i=0; i < vals.length; i++) {
```

```
        vals[i] = vals[i] - minVal;
```

```
    }
```

$O(n)$

```
}
```

```
public static void reduce (int[] vals) {
```

$O(1)$

$$1 + n + 1 + n = 2n + 2$$

$O(n)$

$O(1)$

$O(n)$

```
}
```

Total: $O(n)$

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
            if (vals[i] - vals[j] > max) {  
                max = vals[i] - vals[j];  
            }  
        }  
    }  
    return max;  
}
```

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
            if (vals[i] - vals[j] > max) {  
                max = vals[i] - vals[j];  
            }  
        }  
    }  
    return max;  
}
```

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
            if (vals[i] - vals[j] > max) {  
                max = vals[i] - vals[j];  
            }  
        }  
    }  
    return max;  
}
```

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
            if (vals[i] - vals[j] > max) {  
                max = vals[i] - vals[j];  
            }  
        }  
    }  
    return max;  
}
```

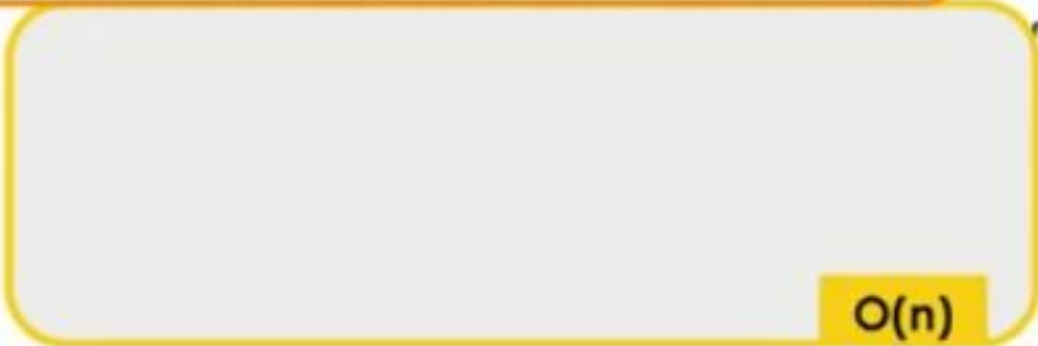
```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
            if (vals[i] - vals[j] > max) {  
                max = vals[i] - vals[j];  
            } O(1)  
        }  
    }  
    return max;  
}
```



```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
              $O(1)$   
        }  
    }  
    return max;  
}
```

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
        for (int j=0; j < vals.length; j++) {  
              
        }  
    }  
    return max;  
}
```

O(n)

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
          
    }  
    return max;  
}
```

$O(n)$

```
public static int maxDifference (int[] vals) {  
    int max = 0;  
    for (int i=0; i < vals.length; i++) {  
          
    }  
    return max;  
}
```

$O(n)$

$O(n^2)$

```
public static int maxDifference (int[] vals) {  
    int max = 0; O(1)  
      
    return max; O(1)  
}
```

$O(n^2)$

Some Examples

Determine the time complexity for the following algorithm.

```
count = 0;  
for (i=0; i<10000; i++)  
    count++;
```

Some Examples

Determine the time complexity for the following algorithm.

```
count = 0;                                O  
for (i=0; i<10000; i++)                  (1)  
    count++;
```

Some Examples

Determine the time complexity for the following algorithm.

```
count = 0;  
for (i=0; i<n; i++)  
    count++;
```


Some Examples

Determine the time complexity for the following algorithm.

```
count = 0; O  
for (i=0; i<n; i++) (n)  
    count++;
```

Some Examples

Determine the time complexity for the following algorithm.

```
sum = 0;  
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        sum += arr[i][j];
```

Some Examples

Determine the time complexity for the following algorithm.

```
sum = 0;
```

```
for (i=0; i<n; i++)
```

```
    for (j=0; j<n; j++)
```

```
        sum += arr[i][j];
```

O
 (n^2)

Some Examples

Determine the time complexity for the following algorithm.

```
sum = 0;  
for (i=1; i<=n; i=i*2)  
    sum += i;
```

Some Examples

Determine the time complexity for the following algorithm.

```
sum = 0;                                 $O(\lg n)$   
for (i=1; i<=n; i=i*2)  
    sum += i;
```

WHY? Show mathematical analysis to prove that it is indeed $O(\lg n)$

Some Examples

Determine the time complexity for the following algorithm.

```
sum = 0;  
for (i=1; i<=n; i=i*2)  
    for (j=1; j<=n; j++)  
        sum += i*j;
```

Some Examples

Determine the time complexity for the following algorithm.

```
sum = 0;  $O(n \lg n)$   
for (i=1; i<=n; i=i*2)  
    for (j=1; j<=n; j++)  
        sum += i*j;
```

Why? Outer for loop runs $O(\lg n)$ times (prove it!) and for each iteration of outer loop, the inner loop runs $O(n)$ times

Big-O Notation

- We say $f(n) = 30000$ is in the *order of* 1, or **$O(1)$**
 - Growth rate of 30000 is constant, that is, it is not dependent on problem size. → **Constant time**
 - $f(n) = 30n + 8$ is in the *order of* n , or **$O(n)$**
 - Growth rate of $30n + 8$ is roughly n . → **Linear time**
 - $f(n) = n^2 + 1$ is in the *order of* n^2
 - Growth rate of $n^2 + 1$ is roughly proportional to the growth rate of n^2 . → **Quadratic time**
 - In general, any $O(n^2)$ function is faster- growing than any $O(n)$ function.
 - For large n , a $O(n^2)$ algorithm runs a lot slower than a $O(n)$ algorithm.
 - $O(2^n)$, $O(4^n)$, $O(2^{n^2})$, etc., are called **exponential times**.
- In general, $O(n^{c_1} (\lg n)^{c_2})$ time is called **polynomial time** (here c_1 and c_2 are constants). For E.g. $O(n^2)$, $O(n^3)$, $O(1)$, $O(n \lg n)$, $O(n^2 \lg n)$, $O(n^3 (\lg n)^2)$, etc.

Polynomial & non-polynomial time algorithms

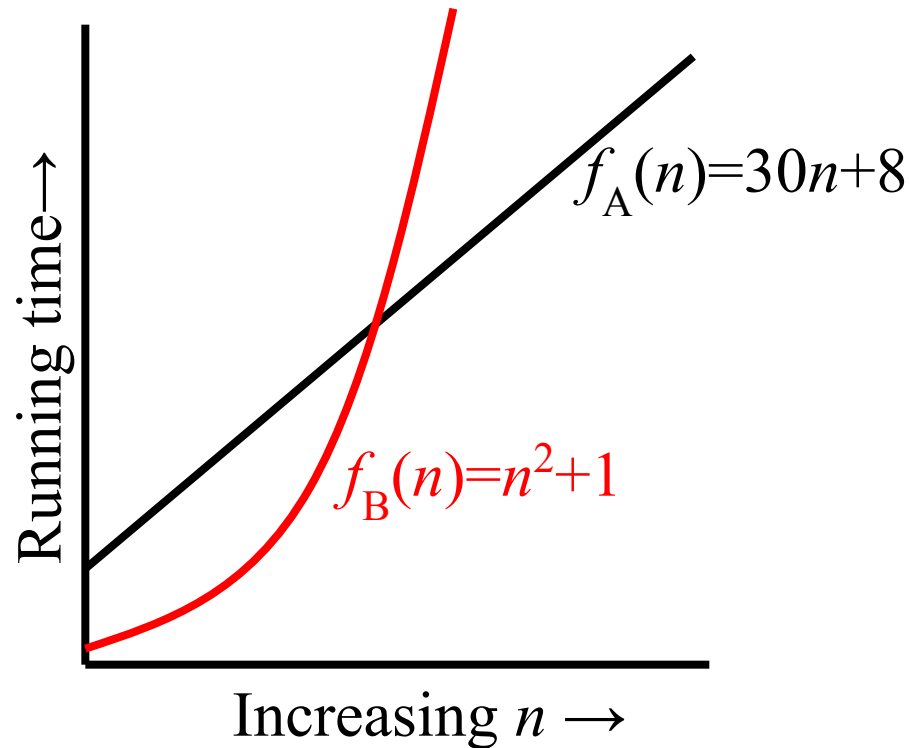
- Polynomial time algorithm: Algorithm whose worst-case running time is polynomial
 - Examples: Linear Search (in unsorted array): $O(n)$, Binary Search (in sorted array): $O(\lg n)$, etc.
- Non-polynomial time algorithm: Algorithm whose worst-case running time is not polynomial
 - Examples: an algorithm to enumerate and print all possible orderings of n persons: $O(n!)$, an algorithm to enumerate and print all possible binary strings of length n : $O(2^n)$
- Theoretically, polynomial algorithms are expected to be more efficient than non-polynomial or exponential algorithms but this is not always true in practice ...

Are non-polynomial time algorithms always worse than polynomial time algorithms?

- Theoretically, yes; but not always true in practice. For e.g. consider algorithms **A** and **B** where
 - **A** is an $O(n^{1,000,000})$ algorithm
 - **B** is an $O(n^{\log \log \log n})$ algorithm
- **A**'s running time is *theoretically* polynomial, so we may expect it to be more efficient than **B** which is an exponential time algorithm. But practically **A** takes much longer time to finish than **B** which is *theoretically* exponential algorithm.
 - So there may exist a non-polynomial time algorithm which is better than a polynomial time algorithm

Visualizing Orders of Growth of Runtimes

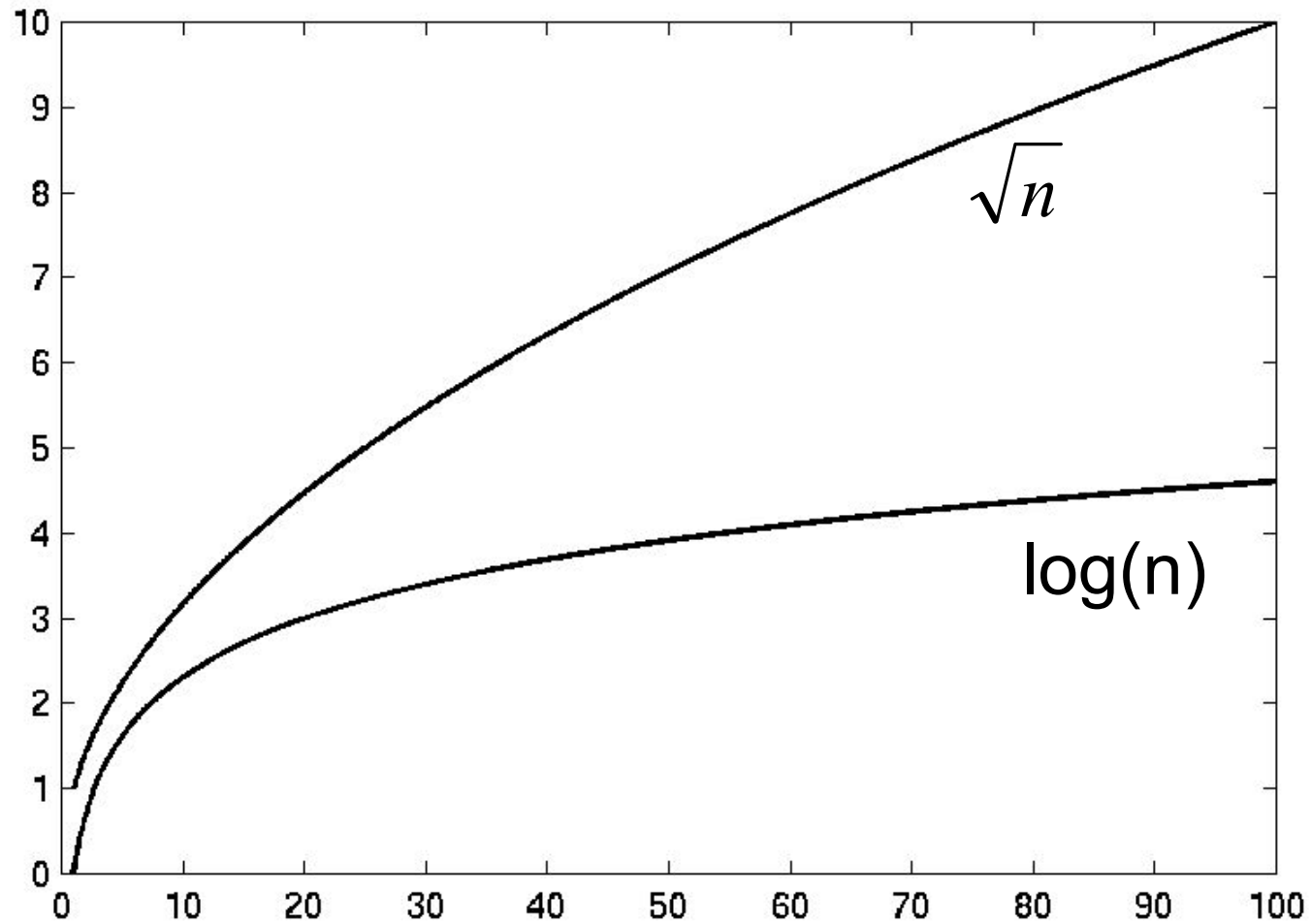
- On a graph, as you go to the right, a faster growing function eventually becomes larger.



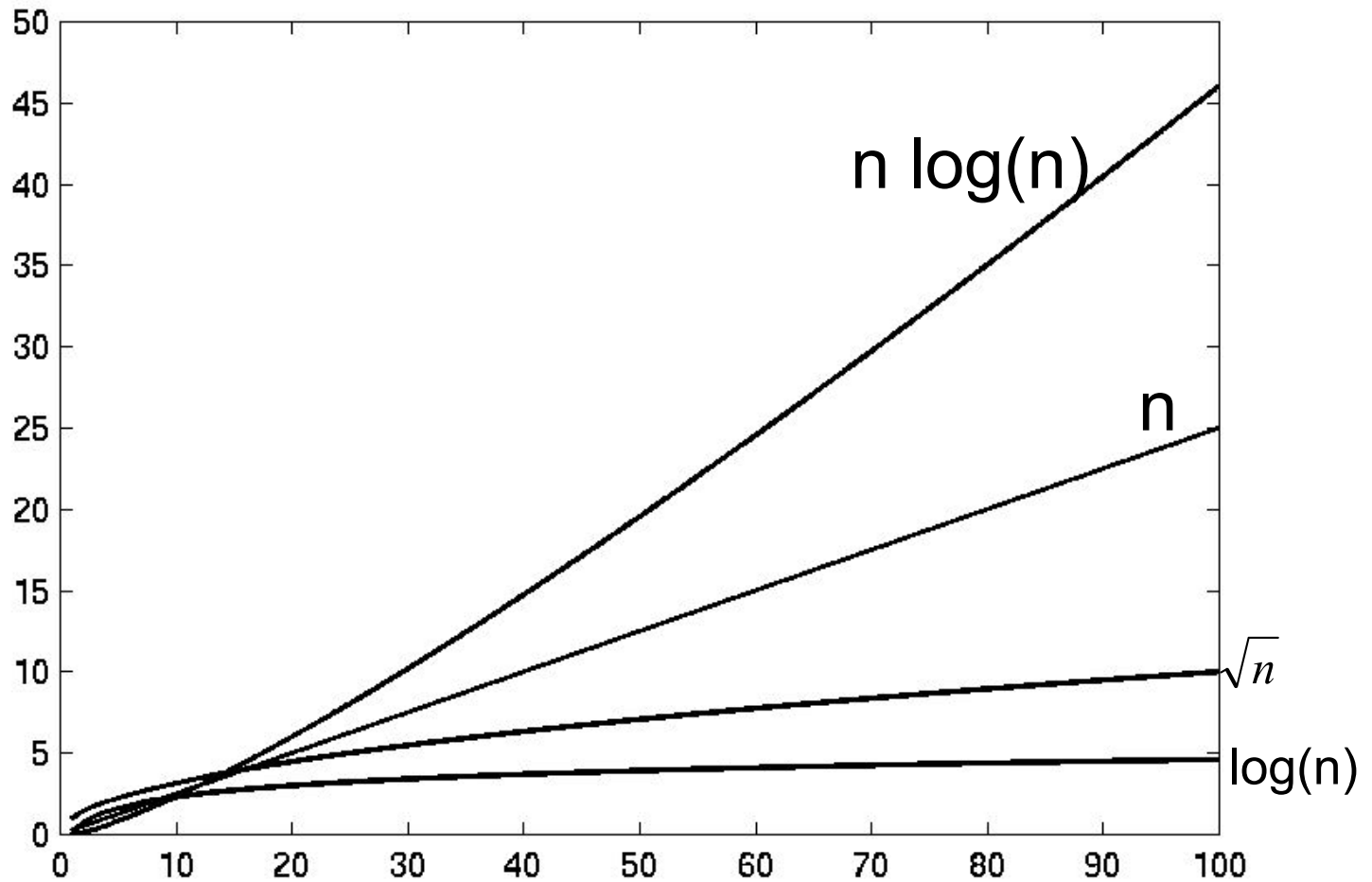
Growth of Functions

n	1	lgn	n	n lgn	n²	n³	2ⁿ
1	1	0.00	1	0	1	1	2
10	1	3.32	10	33	100	1,000	1024
100	1	6.64	100	664	10,000	1,000,000	1.2×10^{30}
1000	1	9.97	1000	9970	1,000,000	10^9	1.1×10^{301}

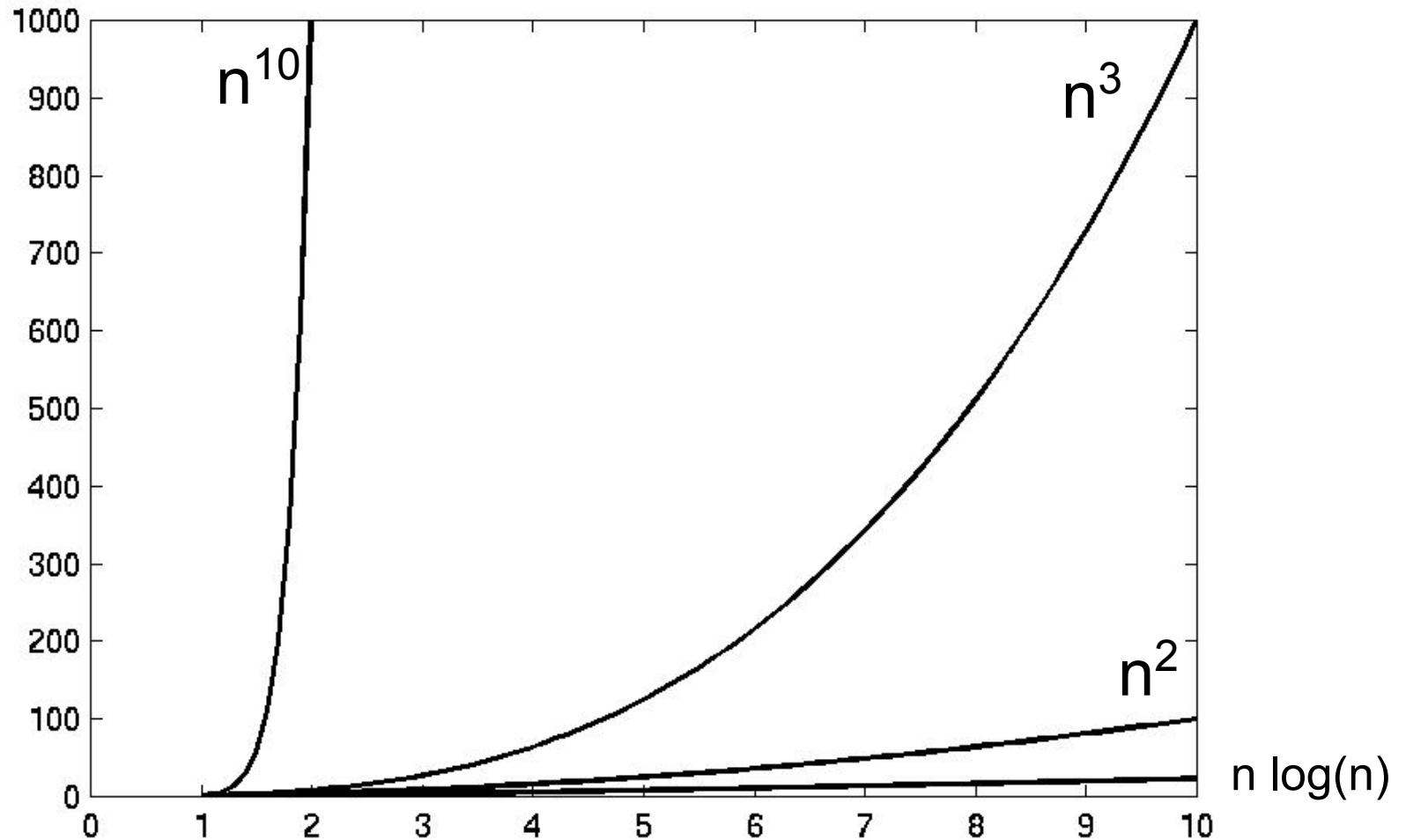
Complexity Graphs



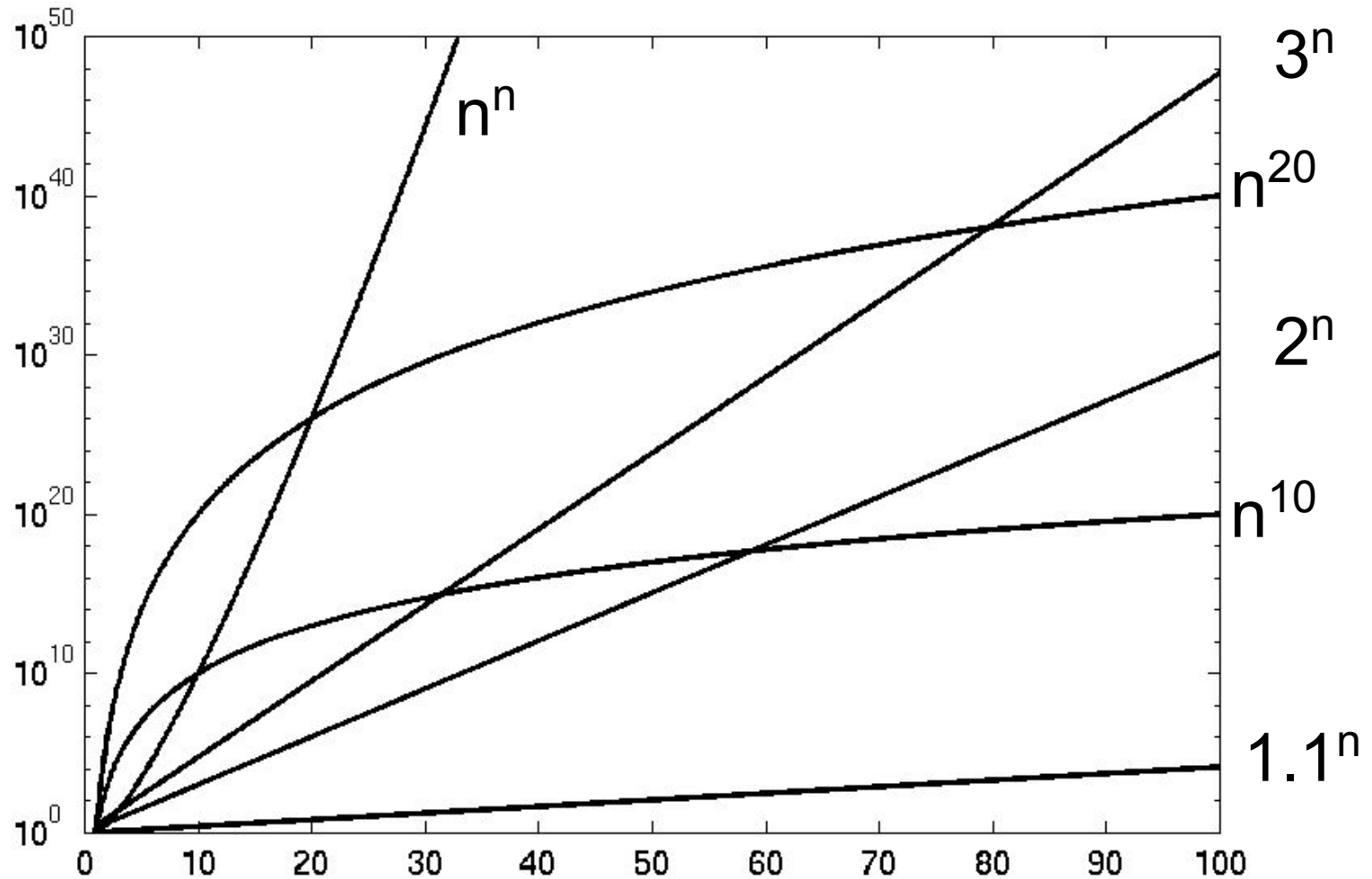
Complexity Graphs



Complexity Graphs



Complexity Graphs (log scale)



Types of Time Complexity Analysis

- **Worst-case:** (usually done)
 - Running time on worst possible input
- **Best-case:** (bogus)
 - Running time on best possible input
- **Average-case:** (sometimes done)
 - We take all possible inputs and calculate computing time for all of the inputs sum all the calculated values and divide the sum by total number of inputs
 - We must know (or predict) distribution of inputs to calculate this
 - Often we compute it on an assumed distribution of inputs using *expectation*, in which case it is called **Expected running time**
 - This is typically calculated to show that although our algorithm's worst case running time is not better, its expected time is better than its competitors

We are typically interested in the runtime of an algorithm in the worst case scenario. Because it provides us a guarantee that the algorithm won't take any longer than this, no matter which input is given.

Insertion Sort Algorithm Revisited

INSERTION-SORT (A, n) $\triangleright A[1 \dots n]$

```
for  $j \leftarrow 2$  to  $n$   $O(n-1)$   
do  $key \leftarrow A[j]$   $O(1)$   
    $i \leftarrow j - 1$   $O(1)$   
   while  $i > 0$  and  $A[i] > key$   
       do  $A[i+1] \leftarrow A[i]$   
          $i \leftarrow i - 1$   $O(1)$   
    $A[i+1] = key$   $O(1)$   $O(1)$ 
```

What is the time complexity?

Depends on arrangement of numbers in the input array.

How can you arrange the input numbers so that this algorithm becomes most efficient (best case)?

How can you arrange the input numbers so that this algorithm becomes most inefficient (worst case)?

Insertion Sort Algorithm Revisited

(best case analysis)

- Insertion sort performs two operations:
 - it scans through the list.
 - it moves elements if they are out of order.
- Each operation contributes to the running time of the algorithm.
- If the input array is already sorted, then insertion sort compares $(n-1)$ elements and perform no movements. So, in the best case, it runs in $O(n)$ time.

Insertion Sort Algorithm Revisited

(worst case analysis)

- The worst case for insertion sort will occur when the input list is in decreasing order.
- To insert the last element,
 - we need at most $(n-1)$ comparisons and
 - at most $n-1$ movements.
- To insert the second last element, we need at most $(n-2)$ comparisons and at most $n-2$ movements.
- The number of operations needed to perform insertion sort is therefore:
$$2 \times (1 + 2 + \dots + n - 2 + n - 1) = n^2 - n$$

So, in the worst case, it runs in $O(n^2)$ time.

Insertion Sort: Running Time

INSERTION-SORT(<i>A</i>)	<i>cost</i>	<i>times</i>
1 for $j = 2$ to $A.length$	c_1	n
2 $key = A[j]$	c_2	$n - 1$
3 // Insert $A[j]$ into the sorted sequence $A[1 \dots j - 1]$.	0	$n - 1$
4 $i = j - 1$	c_4	$n - 1$
5 while $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^n t_j$
6 $A[i + 1] = A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7 $i = i - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8 $A[i + 1] = key$	c_8	$n - 1$

$$\begin{aligned}
 T(n) = & c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\
 & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1) .
 \end{aligned}$$

Here t_j = no. of times the condition of while loop is tested for the current value of j .

How can we simplify $T(n)$? Hint: compute the value of $T(n)$ in the best/worst case

Insertion Sort: Running Time

$$\begin{aligned} T(n) = & c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) . \end{aligned}$$

Insertion sort running time

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

Insertion sort running time (best case)

Insertion Sort: Running Time

Insertion sort
running time

$$\begin{aligned} T(n) = & c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) . \end{aligned}$$

$$\begin{aligned} T(n) = & c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ & + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ = & \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ & - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

and

$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

Insertion sort running time (worst case)

Insertion Sort: Running Time

Statement	cost
INSERTION-SORT (A, n) $\triangleright A[1 \dots n]$	
for $j \leftarrow 2$ to n	$c_1 n$
do $key \leftarrow A[j]$	$c_2 (n - 1)$
$i \leftarrow j - 1$	$c_3 (n - 1)$
while $i > 0$ and $A[i] > key$	$c_4 \sum_{j=2}^n t_j$
do $A[i+1] \leftarrow A[i]$	$c_5 \sum_{j=2}^n (t_j - 1)$
$i \leftarrow i - 1$	$c_6 \sum_{j=2}^n (t_j - 1)$
$A[i+1] = key$	$c_7 (n - 1)$
$T(n)$	
$= c_1 n + c_2 (n - 1) + c_3 (n - 1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7 (n - 1)$	

Here t_j = no. of times the condition of while loop is tested for the current value of j .

In the **worst case** (when input is reverse-sorted), in each iteration of the for loop, all the $j-1$ elements need to be right shifted, i.e., $t_j = (j-1) + 1 = j$: [1 is added to represent the last test].

Putting this in the above eq., we get: $T(n) = An^2 + Bn + C \rightarrow O(n^2)$, where A, B, C are constants.

What is $T(n)$ in the best case (when the input numbers are already sorted)?